
Yours, Mine and Ours

A Presentation to the OCP-TAP Forum 18th December, 2024, 11am PST

Transcript/Recording: [OCP-TAP-02024-DEC-18](#)

[Video](#)

[Slides](#)

Summary

Pat Helland and Daniel May presented on "Yours, Mine and Ours," discussing practical set reconciliation algorithms. They explained how to efficiently find differences in large sets using a variant of fountain codes, achieving logarithmic complexity in set differences rather than set size. The algorithm, which leverages pseudorandom sequences and XOR operations, allows for rapid and accurate set reconciliation. They highlighted its applications in anti-entropy protocols, quorum evolution, and multi-phase protocols. The discussion also touched on the potential for set reconciliation in cloud computing and the importance of time-bounded sets for efficient distributed systems.

Transcript

Ahmad : Hello, everyone. This is call number 120. Today we have Pat Helland who has been building this new existence and databases since 1978 companies including and then Microsoft and Amazon is constantly curious about trends in technology and their implications of systems. Pat has been working on database technologies at Salesforce Since 2012 and also we have Daniel May. Daniel works at Salesforce and has been analyzing and improving the performance of complex systems for more than seven years, spent much of his spare time voraciously consuming technical papers, largely about distributed systems. So today's talk is titled Yours, Mine and Ours, and no with no further ado Pat and Daniel, the floor is yours.

Pat : Hi, I'm Pat Helland, and here's Daniel May. Our talk is titled yours and mine and ours, because this is about set reconciliation. If we have two different systems with different the same set with only slight differences, how can we find those differences efficiently and rapidly? And so we kind of thought that was a fun title. So we allocate the space that's all and so these are our own ideas, and not necessarily reflective of our the position of our employer, Salesforce, and then let both work in Salesforce. So we'd like to do an introduction and talk about the problem we hope to solve and why we think important. And this is driven by a paper that we found at a sitcom in just in August of last year. We saw it a few months earlier, called practical, rateless set reconciliation¹ and we pounded for a long time to understand the paper.

This is our explanation of the ideas that are in it with some different nomenclature stuff. We'll talk about that. Then we're going to say why we think this is super valuable and discuss emerging opportunities and confusion, and especially **we do think this is relevant to time**. We do think this is **relevant to emerging understandings of common knowledge**. Who knows what? When did you know what? And all of those things. But we'll dig into that after we explained this really unusual algorithm, that's why it's so cool.

¹[Practical Rateless Set Reconciliation](#) Set reconciliation, where two parties hold fixed-length bit strings and run a protocol to learn the strings they are missing from each other, is a fundamental task in many distributed systems. We present Rateless Invertible Bloom Lookup Tables (Rateless IBLTs), the first set reconciliation protocol, to the best of our knowledge, that achieves low computation cost and near-optimal communication cost across a wide range of scenarios: set differences of one to millions, bit strings of a few bytes to megabytes, and workloads injected by potential adversaries. Rateless IBLT is based on a novel encoder that incrementally encodes the set difference into an infinite stream of coded symbols, resembling rateless error-correcting codes. We compare Rateless IBLT with state-of-the-art set reconciliation schemes and demonstrate significant improvements. Rateless IBLT achieves 3 – 4× lower communication cost than non-rateless schemes with similar computation cost, and 2 – ×2000 lower computation cost than schemes with similar communication cost. We show the real-world benefits of Rateless IBLT by applying it to synchronize the state of the Ethereum blockchain, and demonstrate 5.6× lower end-to-end completion time and 4.4× lower communication cost compared to the system used in production..

Let's talk about set reconciliation, as I said, there's this new paper from sitcom, and it's called practical weakness set reconciliation. And the algorithm we described is a variance of fountain codes. And if you're a networking guy, you're kind of used to fountain codes. **They are forward error correction** where you keep sending information until the other side eventually says, you know, gee, I got enough. Imagine talking to Voyager spacecraft, you know, like, weeks away. And then eventually it says, you know, that stuff you told me three weeks ago, I actually understood how you stopped sending it again. And so this is also about fast set reconciliation. That's when we doing the thing it does. So you want to take the symmetric differences. If I have a set A with a bunch of items and a set B, how can I find the stuff that is in A and not in B and the stuff that's in B and not in A? And this algorithm is useful for very large sets with relatively small differences. And the point is, it's less than order $n \log n$ work for n being the size of the difference, not the set. That ends up being kind of really fascinating when you think about it.

Well, who cares about sets? Where database is a distributed system? People? What's this set crack? We don't care about that. Well, we're going to argue that this is magic for databases and distributed systems, because you can rapidly and accurately communicate changes in the system. You can repair data discrepancies, you know, anti entropy, this replica is missing, this, that kind of thing. **You can gossip state, and you can optimize multi phase protocols.** Now, interestingly enough, that **optimizing multi phase protocols assumes you can say a notion of time² and what was known in a window of time.** So we have to dig into that. This is relevant to time. Did you want to add anything about time? And then, Daniel, I know we were talking about that.

Daniel: I think that in a lot of our previous discussions that we've been excited about this, because **with the combination of the proliferation of atomic clocks**, and potentially some of these clever algorithms to take advantage of things like epochs or just more precise time bounds. We're very excited about the idea that there might be some non local optimizations that can be made in the distributed system space. So I'm here to explain why you already care about set reconciliation, that essentially, a lot of databases distributed systems, have a recurring flavor about algorithm, which is finding and eliminating differences. So this can show up with anti entropy protocols.

So for example, when, let's say the database, Cassandra, consistently tries to find and eliminate any differences between its replicas, in addition a lot of form evolution, or basically the ways that we evolve our systems by adding and replacing strategically servers, is also predicated on needing to be able to quantify and eliminate any differences in the data between replicants.

Our perspective on this is, this is error correcting by any other name. So there's some familiar techniques to be able to handle this. So for example, Merkle tree is the most classic, which requires, unfortunately, $n \log n$ or $\log n$ round trips to be able to crawl down the tree structure to be able to find what the minimum set of differences are. Because of this, we often see a non local design decision made to force things into being more log based. It's a lot easier to reconcile if there's already a total order of all the events that have transpired in the system. This is an exciting algorithm because it allows us to take a step back and reconsider with a strong reconciliation protocol, what other design decisions can be made.

Pat : What is our perspective of this practical rate of set reconciliation? Goal number one is to explain this thing, which I personally was gobsmacked. And it was just like, oh my gosh, that you can take two extremely large sets like, one application a friend of mine pointed out is, gee, if you have a genome with 300 billion entries in it, and you want to compare genomes, and this is not proportional to 300 billion. Is proportional to the 2000 things that are different. That's astonishing. The paper took work for us to understand it really did, and it took more work to explain it, and we're standing back and trying to make it approachable to understand how and why this works. Goal two is to inspire thoughts about how to use set reconciliation in surprising ways.

²There are 72 references to "time" in this transcript

We want to start with some stuff we've thought about, about what this could be used for, and then get discussions going and brainstorm with smart people about other ways set reconciliation can change distributed systems. What do we mean by sets? It's a set of many items. What is an item? It is a fixed length thing with unique item values in it. Maybe it's 100 bucks, maybe it's a kilobyte. The bid pattern is unique. The set item is unique. This is not key values. You do not update an Item. An Item is not in a set and is added into a set. You can build things on top of that.

You can build a key value on top of it. You can build a database on top of it. But **this algorithm is about the presence or absence of items in a set and their fixed length**. The way the algorithm is described, we know we can extend it to variable length, and we're not going to talk about that. So items are added to a set. They are never deleted and they are never updated. You can make an empty set. You can make a set for stuff you're going to do next Tuesday or next year.

You can make a set for things with this category and that category, and it's empty. And then items appear, but they're never deleted, and then they're never updated. Set reconciliations to find the missing items. If I have set A and I have set B, what items are in A and not in B, what items are in B and not in A? So we want to explain, if you do go to the paper that we're citing the nomenclature differences. So our talk doesn't get you tangled up.

The paper has a set of terms, and we kind of change some of them because we think it's more approachable. The term in the paper is coding symbol. Our talk talks about it as a slot. The term in the paper is a source symbol. Our talk call it talks it calls it an item in the paper, there's an index. We call it a slot number. And in the paper, they talk about subtracting source symbols. We're going to talk about merging slots.

Merging is not exactly subtracting. Merge uses XOR to take the **symmetric difference** and merge actually concurrently adds in some tracks, items from a slot. We will explain this. We will go into it. I just wanted to highlight, if you go to the paper, don't get too tangled up with the nomenclature. Okay, let's talk about our concepts. And we're going to then go in the paper and talk concepts. We're going to do these items and slot sequences, items and slots, how do you encode a representation of a set? How do you find and then how do you decode from the set. And then the magic starts to happen about the set difference and explaining how that works. And then we'll summarize the thing.

In this section, I'm going to give you some inspiration. **XOR can find one missing value in order one time, assuming there's only one**. So imagine Daniel's got a set of a million items. I have the same set of a million items, but I've added one. I can then take the XOR of all of those million plus one items and I send it to Daniel if he xors it with his million boom, out pops. The one that is different. Let me show you how that would work.

Okay, so I have set A, and Daniel has set B, but there's one thing missing. The item B is missing from Daniel's set and is present in my set. I've XORed together everything in my set. And here I've got seven items. It could be a million. It doesn't really matter. And Daniel's got six things XORed together. And now, if we XOR the two of them together, what magically pops back out is the thing which is not in both, because the things that are in both cancel out with XOR. That's the inspiration, the magic.

The problem here is, well, that's fine for one, but what if there's more than one? Is there a pattern for encoding the logarithmic set that can find that set difference in order $n \log n$ of the difference, not the set size? And this algorithm is a variant of erasure coding, specifically common codes forward error correction to for message transmission. And it's kind of a cross between data communications error recovery and storage error recovery.

But the fascinating thing is, we are observing, erasure codes are not just for data you've lost. They can be for data you do not have yet, and that ends up being a different way of thinking about it. So this paper, practical rate of set reconciliation states on the office Yang Jiechi, lad and halaza, and it's called practical rate of set reconciliation.

They talk about the symmetric difference, large sets, small differences in order n , $\log n$ of the difference reconciliation is the worst case of that. So what do we mean about set difference? Server a has set a, server B has set B. They're mostly the same, but there's some differences. Server a starts sending an encoding, an encoded representation of its set, a small prefix of an encoding we're going to explain.

This is an infinite encoding, but you don't send all of it. You send a prefix. When enough of it gets to b that we can tell what if A has and B doesn't and vice versa. It says, stop. So the interesting thing is the symmetric difference, which is the items in one set, but not both. And if there is a tiny difference, you don't need to send much of the encoding, and you don't need to do much computation.

That's kind of the magic we want to talk about, and this is observing the applicability of a research group to sets that don't actually exist yet. Just agree on what the set is. The paper describes an analysis claiming excellent performance, and it points to an open source implementation of the algorithm. And that's just fine if you want to go dig into it, but we're going to point out the things we think are interesting within it.

So what are the goals and underlying techniques? The goal is efficient, set operations and reconciliation, again, two sets of fixed length, items, set A and B, items are fixed length, but they have a different count. B has more than A or A has more than B. The underlying techniques are symmetric difference, and we're going to apply that both for the items in the sets and the items in the slots. I have not explained slots. I will explain slots. We write that as a delta B as the symmetric difference. So the one technique is symmetric difference.

The next one is pseudorandom sequences. I'm going to take an item, I'm going to deterministically map it to a sequence of slots that's infinite, and you can deterministically regenerate that sequence based only on the item itself. We'll explain this. I realize I haven't explained it. And then the third technique is exponential distribution. So as the new slots of a sequence for a single item come they have ever larger gaps. That means you have rapidly dropping probability of colliding with a slot from another item. I'll show you more in gory detail.

First, some concepts and terms. An item is a fixed length unique set element, and we include in that hash code derived from the item for error checking, amongst other things, and you can extend it to variable length, but we're not going to talk about variable it's fixed length. Variable it's fixed length for this discussion. Now, what is the prefix of an encoded set? You take a bunch of items and you're going to encode them.

That encoding actually is designed to go on infinitely, but you don't have to send all of it. You send the beginning of it. The more stuff you put in, the smaller the begin. I mean, the less stuff, the smaller the beginning, the more stuff, the bigger the beginning. And it is a sequence of slots identified by their index. And each slot encodes one or more items, or actually zero or more. We'll talk about that. And the slot contents in the slot index are transmitted in increasing orders, starting with slot zero.

Empty slots cost nothing to transmit because each one's got an index number. You can skip over the indexes. What is in a slot? It's an element of an encoded prefix of a set, including its index.

The contents of a slot with zero or more items, you take each of the items that's landed in the slot and you XOR them together. Remember their fixed length. They could be 100 bytes. They could be 1000 bytes. They could be 100,000 bytes. You have to design that when you're building your solution for this. But you take the items that had landed there and you XOR them together, and that's the item sum, each individual item you hash. So you now have, if you have n items in the slot, you have n hashes of the individual items before you XOR them, and now you have a hash.

So which XORs those hashes? Three interesting cases for a slot. Nothing's in it, no items. It's an empty slot. One is it? It's referred to as a pure slot. Exactly one. More than one, it's an **impure slot**, and we'll be able to tell that difference. When you merge two slots, you're XORing

the contents together, and it can actually increase or decrease the items in the slots that are there. And we'll explain how that works. Finally, there's this per item sequence of slot indexes. I start with an item, and I end up getting these indexes that say, Where will that item land. And those two tricks we use pseudorandom. I start with the item as a seed, so its slot sequence is identical every time I go with that same item. And it's exponential, because we increasingly create a greater stride across the slot indexes by approximately one and a half. Show you that right now we're going to talk about items and slot sequences.

Every item maps to an infinite sequence of slots. What the heck are we talking about exactly one item? I'm going to figure out the slots that it might land in. Everything lands at zero. So that's a rule. Now I use pseudo random stuff. Take the item, I've hashed it, and I use it as a seed to a pseudo random number generator. That means, for this item, I'm going to get the same slots over and over again every time I try.

Started with the item, got the hash, got the seed of a pseudorandom generator, and I'm going to use some arithmetic tricks to get exponential gaps as we go. So now it's deterministic. Give me an item, I get the hash, I get the pseudorandom thing, and I get the same sequence of slot indexes. Now each slot is going to be collided based upon different items that did multiple more than one land there. If there's nothing there, we can tell because the item sum is zero and the hash sum is zero, and so nothing landed there. If there's exactly one, we can tell because the hash is actually the hash of the item sum.

So we know that only one thing's there if the hash sum is not equal to the hash of what I have for the item sum. Then something went wrong and didn't go wrong, but more than one thing landed together and was XORed into a much the larger the slot index, the fewer the items statistically land. More become pure, and more become empty. The beginning of an encoding is very dense. It's got everything, and then just more and more the tail of it becomes sparse. So we're going to show you how that works. Here. It's an infinite sequence of slots per item. Each item maps to an infinite sequence starts at zero, and then you have to say, how far do I go to the next one? How far do I go to the next one? So I'm going to calculate a gap by taking the pseudo random number generation of the item.

Talk about that in a sec. Pseudo random number generators, the seed equals the item's element, and it gives me a sequence of random values from zero to one, and I square it, and I take I square root it, and then I won over the square root now minus one, and that gets a distribution centered on one and a half. I use that to calculate the next index. So I land this thing, landed at zero. I had one, and then I multiply it times a gap, which is this stride of the distance. So each time I get a new one, it's going to give up one and a half times as far away as the idea. So the gap distribution impacts the coding density. One and a half is actually close to the theoretical Shannon limit, but grounding it to one and a half is good for computational efficiency. And there's a lot of fun material in the paper on that Shannon density in the Shannon limit of theoretical communication. But let's motivate it.

There's a PowerPoint description to kind of make it make sense. Each of these columns, 0123, the columns represent slots. And this is goes up to slot 42 only because I wanted the PowerPoint to be pretty and I ran out of room the PowerPoint I'm going to encode Item a, I take Item A, and I hash it, and I feed it into the pseudo random number thing, and out pops each time, more and more different slots. So everything lands in a the next one for a happens to be in slot two, so it's so zero in slot two. And then the next gap is to slot five, so it's a bigger gap, and then it goes from five to 10, and then it goes from 510 to 19, and then 19 to 27 so the strike is approximately one and a half times. But there's randomness in there, which is driven by the pseudo random string, so it's repeatable every damn time.

So no item B, same thing, but it's going to land in different places because it's got a different pseudorandom number seed, and so it's going to land in different spots, sometimes bumping into A, sometimes not. Add C, and it's the same thing, sometimes bumping into the ones that are

there, and sometimes not. But they get farther apart statistically, the odds of bumping into things drop as you get to bigger numbers. Item D, same thing, always in zero. And then where it lands depends upon the luck of this, how the pseudo random number thing generates it.

These are two infinite sequences. These are possible places that an item may live if you need it for a prefix. Doesn't mean it necessarily does. It means it might if you need that to be encoded. So here I have slot Zero has got all four that I put in it. Slot one's got two slot, two's got two slot, three's got C and D's got two, nothing landed in four. Slot, five's got a two landed in six, one landed in D. And so the essential properties of this are pseudo random number per item is a seed, so it's deterministic and repeatable per item, an exponential increase of slot indexes to the probability of collision drops

Daniel: rapidly if an infinite sequence doesn't necessarily make sense at the first here, there is one perspective that you can take on this. If you're familiar with Merkle trees, you can imagine that this zero sequence is capturing the sub tree at the root, every element that's in the entire Merkle tree. As we walk down the slots, it's like we're moving breadth first across the tree. We're getting smaller and smaller subsets captured as the XOR of all of the items. What's different here, though, is that instead of being structured like a tree, where every single sub tree is just Greek. This is instead randomized. It's basically like we're getting a randomized sequence of breadth first searching of a tree, where sometimes we see elements show up in multiple sub trees,

Pat : Which is not what Merkle does. They don't show up in multiple sub trees in Merkle so there's a notion of a generator. How can I find the items that are sharing the next slot? So if I can find the items sharing the next slot, I can excerpt it together, and I can make the next slot okay? So I can generate slots in the prefix. So think of an event horizon with the set of items in the next, larger slot per item, generate the slot index of the sequence and keep a priority queue of what's next. So if I've got four things in slot zero, when I put them in slot zero, I calculate where's the next place that land. So A is next place is slot 2d, is next place is slot 1c, is next place is slot 1d, is next place is slot three. So I have a priority queue of the next stuff to do. So now I'm going to move to the next slot with some generated items. Add slot items into it. Well, I had B and C, so I add them into slot one, and then I move them forward in the priority queue.

And here I go to A and B, and then I move them forward. And as I continually generate that priority queue, things get less and less dense, and they move out. And it's quite easy to think about generating the prefix pre canned and then the ability to have the slot generator to move forward if I don't have enough in my prefix. So I can optimize this in multiple ways, but you can generate the sequence of slots and where things land, and push them forward to the next is a priority, two of the next slots. This is how one can think about taking a set of items and generating an encoding of where they smash together. So items and slots, let's go into items and slots. And what does that mean? So if I envision a slot in a sequence, let's just go to slot eight. It's got one thing in, it's got C and it's got the hash of C. So it's got the item value, it's got the hash of the item and I go to slot four, and it's got two things smashed together. So I took the payload of the item c and a payload of the item D, XORed them together, whatever fixed length it is, it's fixed length, and then the hash of T, XOR with a hash of D. So that's what landed in four and in zero.

I got lots of crap smashed into zero. It's denser because you would go so that's a way of thinking of the slots in the items. What do I mean by merge two slots? So this is a typo. It should say merge combines two slots by XORing the elements. So you XOR the item sum, and you get a new item sum. You XOR the hash sums, and you get a new hash sum. So if slot X and slot y are two things, I merge together. X has got items A, C and D, y has got items C, B and D, and I'm going to XOR them together. Well, what happens is C and B cancel each other out, because the bit pattern of the item of C XOR is into oblivion with the bit pattern of the item of C similarly b. So what I end up with is the combination of A and B XORed together. So this merge actually removes things, removes items that are in both, and adds items that were only in one and so it has this interesting behavior of adding and subtracting things from a slot as you create a new slot by

merging two. So let's talk about encoding and decoding. I want to encode a set of items, one at a time. So I have an empty set. I add one thing in, and then I get it there, and I add another, and I keep adding them one at a time, start at slot zero and merge the item into slot zero by XORing the item and XORing its hash. If that's pure, if I've got only one thing in it, then I exit the loop. If it's not, then I go to my slot sequence, go jump into the future to a new slot, and keep merging until I find a slot where there wasn't something there first, and then I'm done. So let's start out. Let's assume I have A and B and D already present in an encoding, but I'm going to add c to the encoding. So these are the places that C might live, and that keeps on going.

This is just the beginning of the places C might live in its sequence that pseudorandom generated. So I add C to slot zero, I XOR it in, and I've got it there. But by gosh, that was not pure. Where's the next place slot four? So I go in there and I XOR it in, but gee, that wasn't pure. There was already something there. So I keep going. Where is the next place that I should put it? Because four was not pure. Well, the next place in the sequence for C was slot eight. So I go to slot eight, I XOR it, and I say, Wow, there was nothing else there. It is now pure. I have added C to the encoding. How do I decode? I decode a set by peeling because the encoded slot has set like many slots, some empty, some pure, some impure. I just keep going until I find something that's pure. So you're sending Daniel, sending me a prefix of an encoding. I'm trying to take it apart, find the items in it. Have to wait till I find one that's pure. When I find one that's pure, I'm going to decode it incrementally by generating a slot sequence and going back and then move backwards until I get to slot zero and take it out. So I'm going to incrementally peel it out until I get back to zero. So I'm going to look for pure slots where the hash sum equals the item sum, peel it back by merging it in and then continue until I get an empty one. So let's show this. So I'm going to receive an encoded set a slot at a time. Well, I get the I get slot zero. It's got four things smashed together, and I'll remember that slot one, and that's not pure, right? Skip slot zero because it's empty. That's cool, right?

And I get to slot three, and it's got it's not pure. Slot four is not pure. Oh, my God, wait, wait, wait. Slot seven arrives because I skipped five and six. Slot seven arrives and it has a pure item. B, that's cool. Now I can start making progress. So I peel, remember, B, remember, is pure. I got B, B, I see it in clear text with the item value, and I start peeling it from the encoding. So remember that I got B, I go and I want to follow the slot sequences for slot B. And these are the places B can land in its infinite sequence. So I'm on seven. I started at seven with it being pure on seven to go backwards to zero from seven, so I peel it out of seven. Now it's gone. What's the previous place for B? Oh, it's, it's, you know, slot three. I skipped slot four.

I go to slot three. I peel it out of there. And it's not pure yet that I see that peeling b out left a pure. So now I remember that a is now pure in slot three, and I continue to peel B from slot one in slot zero, but I am not done yet. However, I've now got a pure A in slot three, so I can peel it. So I start peeling B. I stop peeling B at slot zero, but I know slot A is pure in three. Let's peel it from three. So I peel item A out, I XOR it out of slot three, and I end up with an empty slot three. That's cool. XOR out of slot one, item, empty slot one, and I XOR it out of slot zero, but slot zero still got stuff. So I stopped peeling a but I'm not done yet. I don't have anything pure slot zero and slot four got multiple things. There's more than one in each. So I got to keep going for a while. So let's take more slots, and I get slot eight. Slot eight is a pure C. I remember it. I peel it out of slot eight, I peel it out of slot four, and I have a pure D. I remember D is pure in slot four, and I peel it out of slot zero, and now I can go and repeat the process to take D out. So finally, peel D and notice the set is decoded. So I've decoded an encoding of this set. Now let's get to the magic. Let's show how you can do set difference in order, N , $\log n$, and how this is wicked cool. You want to say anything first, Daniel?

Daniel: No, go ahead and move on to the next slide here, and then we can talk through it with the demonstration of the magic.

Pat : Cool. So let's take the set difference from a prefix of encoding. Say, I'm sitting on server B, and I got set B, and I have A, C, D, E and F, G, I got six items in there and set a, you know, Daniel's got set A, and he's got four items. Some of them might have. Some of my don't. I don't have B, okay, and, but I have the other ones I have, but that, yes. And so he sends me his prefix. And so I take slot zero, I XOR it together, but I still have it's not pure, but it's got B, D, E and G. So if I want to look at reconciling slot zero, what does that mean?

Daniel's four and my six have an overlap, but things that are different, okay, so if I XOR them together, what I see is the difference is there's four things that are in one set and not the other. I got four things to find as we reconcile, turns out as B, D, E and G, but I can't tell from that munched up X or yet. So that's what I'm looking to get. Is the difference? So now the slot one comes and it's got a and I've got a E and G. So I XOR them together, and I end up with the difference between them is E and G. As I look at the set difference, B comes and it's pure.

That's a pure thing, because B and C come from a, but I XOR with my C, and now I've got a pure B. So what happened is, the things that were in common, the C, in this case, they're disappearing into oblivion because they're XORed away. Now I received after X or in a pure B, so I can take that symmetric difference, I can peel it out, and now it's gone. Okay, but now I don't I still don't have anything pure, so I got to keep cooking. So what happens is I get slots number four or three rather, and I XOR them together, and what's remaining is the difference, which is E, because A and C and F are in both, and they XOR away.

So now I take the E and I peel it and I take it out, and that's cool, but now I've realized I've got a pure G, so I can peel G and take it out, and then I realize I have a pure D. So in a mere four slots, I was able to peel the stuff that was different. And it wouldn't have mattered if the sets that were in common were larger. That's the difference I can do. So I have four items that emerge, and I have peeled them out by X omega together.

So remember how we had one item of difference when we XORed a million things together, a million items are saved. One is different, the one that was different popped out. That's great. Remember that that was done in order one because of the difference. How about finding three items of difference, slot by slot, XOR the sets? Turns out that the time to do that is order n, log n of three, or kind of six to 12 ish. And so you can see that the three items are different. So if I'm doing this together, and these sets are huge, there's an infinite amount of other crack that's in common.

I've got three things that are different, and it should take me about n, log n of three. So I get slot zero, I get slot one, I skip slot two, I get slot three, and there I can begin to peel. So I begin to peel C, and then I have to keep going, but those are empty, so I can skip them, and then I get b, and I peel it out. And that this particular PowerPoint example ended up with about order seven, which is about n, log n of three. So the bigger difference is it goes up by order n log n of the difference not the corpus. You can have an arbitrary amount of crap that both of them know and you don't see it. Okay. You want to say something? Sure,

Daniel: Absolutely. So on the previous slide, putting this back into the tree. Contextualization is that, unlike other structures, as soon as you hit any layer that differs by one, you can now recover the value. What this means in practice is that you don't necessarily need to do multiple round trips to recover more data. In this case, you just need a good prediction of how much of the prefix to send in the first place. So for example, as a sender, I might make a pretty good estimate of what the difference is between me and my receiver. Let's say I expect that they are probably they might have lost up to 10 values. That's my best case prediction with 99

Pat : Actually, it's not even a round ...

Paul: Yeah, **It's not even a one round trip.**

Pat : Yeah, **exactly. It's a unit.**

Daniel: And then he's got it, yes. And then there's a variety of other techniques you can layer on top of this to do even better for follow up estimations. But this is, this is the exciting part of this algorithm is that basically you can, you can throw bits without knowing what the differences are, and the receiver can recover up to some amount of differences. They can do. They can treat this like an error correcting problem. Magically be able to recover the difference on their end. Their end without having to have any additional coordination.

Pat : And the math, if you have a pretty good guess, you can just talk about the probability that that's enough, right? And it doesn't have to be perfect, because the other I could say, well, I didn't quite get enough anymore, and so that's just a little bit of tax on top of it. But you can just do a very proactive, just reconciliation.

Here's what I got, here's what I got, here's what I got. And you can, you can actually fine tune into guessing with different guesses, and just almost always make it be once, one blur by the unidirectional send to make sure that knowledge is communicated. And that's pretty cool. I get it right. [Daniel, yep]. So summary, practical rate with set reconciliation. New technique for set difference, low cost set difference in time less than order n , $\log n$ difference not n , $\log n$ of the set size leverages three concepts per item, infinite slot sequence, which is exponential and pseudo random, always start at zero XOR and taking that symmetric difference per slot. So the items in both slots are removed after XOR, and then pure slots start peeling items.

So you peel the XOR to remove the impure ones. Each set has an encoding created for it. It's a flexible encoding. You can pre calculate the beginning of it, so you don't have to do this generator from the beginning. You just do calculated at once. But as you need more, you can then take that generator and cause them to come as you need them. But pre calculating is a great trick, like here's my first 100 and then that just works. And if my first 100 if I add more items to the set, I take the existing set and I can add those items on top of it. And my first 100 are pretty easy for me to calculate as I add items to the set, 100 being an arbitrary number for reconciliation in either or both directions, two different sets can be coming at each other, and you just kind of mash them together.

And it works if a prefix transmission. So it's a prefix of this infinite coding, and enough the simple receiver says, stop. But again, you just guess, and you're pretty good if you do it right, and the receiver reconciles the prefix with a local set, which rapidly finds the difference. Each set can send batches concurrently and it reconciles correctly. So this is pretty powerful. Now we think there's emerging opportunities for this that are really great. So, you know, Danny, you want to talk about the Gauss se protocols? I

Daniel: didn't know this shit? Yeah, absolutely that. And I think this is one of the areas that would excite this group as well, because what's critical to this is agreeing on a set without knowing its contents. And to do that is to use clocks and time bound that we can all agree that the set should be what we received in the last 10 seconds with a boundary condition of arbitrarily accurate nanoseconds. And it's this wonderful pair of hammers that we can hit a lot of things with, including efficient bugosa protocols. Once you built these sets, you can send them to many participants and be able to come to agreement across the distributed system very efficiently.

Pat : Who got what in what period of time is very interesting. Now there is an algorithm variant that the talk the paper goes into in depth, which I think is really cool. I would probably redo the slides and make it included next time around, which is time Account field, so if I know how many items I put in this particular slot, and I receive another image of the slot in another pre in a prefix encoding.

And the count is different. You can subtract the count of what you're receiving from the count you have, and you can tell if the number if you end up getting down to one in that count, if it was negative, then the item that you're receiving that you didn't have before came from the other side, if it's positive, then I already had it.

So that's a really interesting thing, which is, as you gossiping, sets a new incoming item when you didn't have and what you've been listening to so far ends up being negative, and so you put it there. Oh, shit. So now you can detect what's arrived brand new into my server.

So I have 100 replicas, and everybody's just talking shit to everybody gossiping. Each server will exactly one time see the new item pop up to be processed. Now the ordering is not the same necessarily in each one, but they pop up efficiently, super efficiently exactly once is you gossip. That's pretty game good. That efficient gossip is, I think, going to change distributed computing over time. It's really cool. Then you got the thing you want to add

Pat : Let's jump into the next slide. Deadline, base, go for it. Yes.

Daniel: So this is the crux of why I think this, audience would be interested, because really, this changes that, that worldview we need to order everything ahead of time to have efficient reconciliation, and instead goes into a more partially ordered worldview, which is we would like to be able to distinguish between things that happen in the future in the past, but we're okay for some of these things happening concurrently. We can do use error or these error correcting codes to be able to recover a perfect view of the world of some previous epoch. But this means we shift into a more a partially ordered world, where the two tools at our disposal are deadlines and error correction. And how are we going to do this? Any other means than having efficient and accurate clocks

Pat : If you want to adjudicate what made it in what epic, what window of time, and you want that perfect truth that this is exactly what made it, this is exactly what didn't you can choose to have a timekeeper server, and it takes whatever you got when its time is in this window of time, and it closes that set, closes that epic at that point in time, and says, here is what made it, and here's what did that's one technique, if you want to avoid jitter on that you can actually make the timekeeper be a quorum of a set of servers. And you can actually say something made it, if it made it into two of three or five of nine of the servers that I have.

And then that defines it made it or it didn't. That gets fuzzy. If you only have a subset of the knowledge, because you have a you actually have a tri state. You know the things that made it for sure, absolutely, you know the things that didn't make it for sure, absolutely. But if you can't talk to enough servers, you might have a little confusion around the edge, but it's super useful for many algorithms.

And you can define the bands of time, like I do a millisecond band, and then I've got the 10 millisecond, you meaning of those bands, and I've got 100 millisecond, you meaning of 10 bands. That class of things you can think very differently about deadlines that **allows you to rethink common knowledge**, which is a fascinating thing. Closing a deadline based set is I know what you knew. I don't know what you know because time's gone by, but I know what you knew up to that deadline, and you can adjudicate transaction commit with optimistic concurrency control. And you useful for many distributed systems algorithms. I know Daniel wants to talk about that.

Daniel: I think this is the one of the big hammers that we've been excited to run around and hit things with, because you can recapture a lot of existing protocols. Like a good example is something like two-phase commit, which we'll talk again a little bit more about in the next slide, as a common as a mutual knowledge problem, that what you want to do is make sure that all the individual servers don't act unless they have knowledge about the state of the others.

But instead of having this be done on a piecemeal basis, the normal way means that you send out, you know, your your your first round of communication, and then your second round of communication adds the information about what all others that are present. You can have an inferred batch operation, send all of the information to all of your servers to say, Oh yeah, we would like to our goal is to commit some value, and then you perform a round of reconciliation. Any disagreements will fall out from this, but all of the agreements, they just go away next

Pat : door is it is all the shit that you agree on just goes away fast, and the only thing you have to look at is the crap you got to clean up, which is the stuff that didn't go away fast. That ends up being super interesting and powerful. I want to point a different thing out. Okay, this opens up a whole different style of computing in cloud computing.

Cloud computing is weird. It's fun. It is weird. You separate the server state from the store from the compute, and storage are separated. The classic way we do this is you take a set of virtual servers, you find compute, and when they make changes, you're logging the changes to things like EDS or other things like that, that are capturing a log based sequence of transitions this server took. And so you can think of this virtual server is freeze, drying and popping up somewhere else.

Today, in the cloud, you typically have *log* based state of the separate compute, storage and compute, and the single history of the server can be reconstructed on a new compute instance. I think of this as like Captain Kirk in the Starship Enterprise. You're talking to James D Burke, and he says, being me down to the planet, Scotty, right? And then he has a couple of seconds where he kind of goes into suspended animation, and then he appears down on the planet with all of his memories and all of the decision making, all of the algorithms that are going to decide what he does in response to a circumstance. Just moved, but he stopped, and he moved elsewhere. And he has a total order of memory and personality to beam him down.

That's what we do in the cloud. We emulate distributed, classic distributed computing with a server in a closet with direct attached storage. We emulate that separating compute and storage into a log based thing with a total order on a per server basis. If you rethink the algorithms to be set based, then tomorrow items arrive in a set in any order and its deadline based close can provide order if needed. And so set based cloud implementations can be more responsive and robust if you rethink things as sets and arrival of items into sets.

Daniel: Yeah. So let's bring this into concrete terms. As you can probably tell from this presentation, this is a hammer that got us both very excited, and we've been running around trying to find what nails we can hit with it. One of the first things we brought up with this was anti-entropy, because a lot of existing systems that one of the call outs here was Cassandra, but a variety of different systems that use this, and it's very exciting to be able to reapply this is potentially more efficient anti-entropy systems.

The other classes is quorum evolution, which, which we can use to use these algorithms to do standard things like catching up replicas from a restored snapshot, or potentially even more bizarre things like doing uncoordinated data discovery, where you throw in a radical cut and you find out what it knows the rest of the system doesn't. And then lastly is the one that we alluded to here, which I am definitely the most excited about, which is re implementing multi phase protocols using inferred knowledge, where basically that you can infer agreement because all of what was x sort of way you can essentially patch this on top of existing systems that are don't have good consistency semantics, and have a one round version of a one round coordination to say,

Great, let's assume that there's semantics, there's inconsistency between our replicants. Let's resolve that and say that we have a line of epoch that is consistent, and then we have a epoch line that is inconsistent. This very common knowledge focused view of protocols and inferred agreement is really exciting, and is fundamentally the combination of error correction and time bounding with clocks.

Pat : And it's super, super interesting to think about that as a variant of consensus or in the two phase commit all these things. Yep. So conclusion, in the past, databases adjudicated change, arguably, now, with sets in this fashion, they can discover change, not just the transaction load. I'm going to say efficient Event Processing is emerging as important pattern. Vibrant sets of knowledge rapidly visible to many servers. Let me just throw a random crap out here.

A Large Hadron Collider generates like millions of things that are not relevant. You want to find the one in the millions that is worth paying attention to. So as these events fly back by individually,

you can drop them into a different set, which is this one has these characteristics. Now you put it in a different set, but all of the you can do that redundantly filtering. There's a ton of things you can do, if you just think about events and items and qualifications of what fits into what set, and the nature of defining empty sets that make sense is going to be bumped and Fast, fast reconciliation can change the nature of dynamic systems. And finally, I'm going to make one other point. I'm a database, distributed systems person, but I was not beyond looking at Sigcomm to see what interesting papers were there pay attention to other things in the industry. So now let's have a discussion. Let's stop presenting. Let's start talking. What do you guys think you do with this crap?

Ahmad : Thank you. All right, so it looks like the condition to us. Question is, turn the camera on me. Game On.

Hesham:I'm thinking it's all great, but I'm thinking of practical implementations in policies. This like insecurity, for example, fees or something like that, oh, yeah, like for example, when you use something like ease of resource management protocols, maybe it can make use of this.

Pat : I don't disagree. I do agree. We have this crazy we don't know what to do with this stuff, and so we're being open about our ignorance and open about our excitement, about the fact that there's there's gold in them hills, and key reconciliation, sharing knowledge that's phenomenal.

Hesham:So if this is the case back then, I would think RPKI portion is a good example of to apply the algorithms you described.

Pat : I don't know which method

Hesham:Like Roman P like Peter Ki, oh, yeah, I think this would be good application for for what you described.

Pat : I had no problem with that. Completely comfortable with that. That's just one of a bunch of things people to be excited about. How to do this in different ways.

Hesham:Yeah. Excellent, excellent.

Anjali: Okay, so I want to add something. So for me, if I see something and I get reminded of old stuff that I've seen somewhere. So we learned about this thing called katapaya Being system, and I put a link here. And this comes from, you know, an old system where they used to have Sanskrit encoding of knowledge, and that was, there was an error correction system built into it, and that is something which I see here that you were just presenting the paper.

So I haven't, of course, looked at all the details, but I got reminded of it. Sorry, I just put it here. It seems this was similar to what they were doing when they were encoding knowledge, or certain things like the value of π , which is encoded in kardakari to, like, you know, hundreds of digits of π . And so they would just encoded it in a way that if there was some thing that changed, it would just not add up to the rules, and so they could detect it. So, yeah, no, I Yeah, error correction went this far back.

Anjali: Yeah, they had right, and that was why they were like, so particular about, like, okay, when you chant a particular science mantra or something, it has to be set with the right tone. And, you know, all those things, because there's that's encoding title is just going to be wrong. Absolutely.

Pat : My brother Andy annihilated my Hawaii by vacation once with and it was joyous. I went, I spent five days sitting on the Lanai at the condo we rented, **reading about fountain codes and Ruby transforms and how to do forward error correction and messaging.**

I was just gobsmacked about how cool it was. And this is, this is the same and different. This is even more but whole notion, and we haven't talked enough about what happens if you send a

prefix of an encoding and some of the standards we have not thought enough about, that this is beautiful.

Anjali: This is just beautiful. Like, you know, this opens up so many possibilities of thinking, you know,

Pat : I personally love the fast and loose interpretation of this, where there's a variety of different databases and distributed systems that work most of the time that they are not necessarily correct protocols, but you know that they solve somebody's real problem, that it would be really fascinating if we can treat this sort of like how error correcting codes are treated in networking words, instead some patch that you put underneath existing systems where that or We have a clean layer of abstraction to basically fix systems that work most of the time.

Pat : But I'm fascinated by building a control plane for a cloud system. And all the just things you observe are just propagating accurately, but not in order, not in order, but accurately they're going to be in this you can even time bound their arrival by dealing with what does time bound means? Means this server knows it's seen everything up to this point in time, whatever that means. But thinking that through how time weaves together with this, we are just at the beginning of this, and this is not deeply related to our day job, so we get distracted and find time on our on the side to work on it. But it is exciting enough. I want to find some nerds to bond with on it, if that makes any sense. Okay, other nerds talk up. I'm gonna speak up. Who else?

Bijan: I came in halfway, but looking at what you were showing, it reminded me of a couple different things, like I worked on network coding and, you know, some other things that really struck me as kind of a from the same tree, almost, because a lot of that is set theory, and a lot of it is kind of, you know, gathering information and group Blitz, if you will, and so on. So I but what I was ... if I'm surmising correctly, the information rate improvement here is pretty high. There's a lot of gain, right? So

Pat : the paper digs deeply into that. I was a little less stressed about the density of the encoding myself, because I wanted to see more about how to think differently about sets. But there's a ton of stuff in that paper and say, Come, you know, called practical rate of set reconciliation, about how to titrate towards the Shannon theoretical limit of information and coding. Daniel understood that better than I, because he's not as ignorant in math as I. He knows a lot about math and I don't.

Bijan: That's the opiate of the people like me who like to go and develop protocols. It's like, hey, there's a there's a pot of gold at the end here, or very small pot of gold. But yeah, so yeah, that's the thing that caught my eye.

Pat : I'm looking Forward to integrating the theoretian and theoretical information, and they are a question by any means. I personally am excited about how to use it in a way where the systems are interpreting their communication differently. But that doesn't take away from what you just said at all. Go read the paper if you want to go deep into that.

Daniel: It's also the case of the algorithm, that the specific algorithm that was used is likely in a very large family of them, because it's not that different from you know, as you mentioned, the common codes are other error correcting codes that exist. It's using the same concept.

The main philosophical difference in the if you're setting out to design a protocol is that instead of treating this like a recovery problem, where basically that there was an original signal or an original set of bits that were written to a hard drive that you are going to then have errors accumulate, you want to recover instead of treating it as there's some mechanism that we used to agree that the right answer is the union of all of our sets, or really any operation that is that.

But the point is, is that you come into agreement on what the correct value is. It may not have been original version, an original encoded data, but you can still use error correction to basically Marshal your system towards that target. But it's it's all the same mechanisms.

Pat : There's a whole world of declaiming future, non existing sets, and the metadata exploration and the ontology of what it means to agree in advance that in the future, this is the set where these things will land, and that's set with time that's set with filtering and characteristics, that's set with sets with whatever damn properties you want to define. And there's a an unexplored space of thinking about empty sets that can be filled over time and then propagating when they do. I haven't figured it all out. I'm excited about it. And I scratch my head sometimes when I'm wandering around and pissing my wife off as I look at the ceiling.

But this is a thing that rattles in my head. Is, **what is that ontology?** What is that metadata? Exploration? How do we create an open ended thing to think about that? Did that make sense when I said it both time and space? And, you know, go ye forth and view it as a tensor space for God's sake, of sex, and think about what gets dropped into. What did my imagination make sense to yours? Hey, Mata, you joined. Thanks. You pinged me earlier. What do you think of my own drugs? Or should I be? Pardon me, I mean,

Mata: I mean, I found it really interesting, and thanks for sharing the link. I mean, this is super interesting. I think about these things a little bit more these days, but I think it's very interesting to think about what you said there, like having potentially empty sets that we can fill in at a later point of time, or propagating that forward. I think that's exciting to think about. It sort of reminds me a little bit, at least, like the notion of it, the one of the more recent works by Berkeley and Leslie it come and CRT on, oh

Pat : God, you know that face to function that I did back in back in the day,

Mata: yes, yes, yeah, yeah. I think that's it's just nice to see things connected page. So excited to see I think both. I wrote a

Pat : little bit about that in the paper I did call don't get stuck on the COVID, which talks about the ambiguities of consistency, convergence and influence. And there's just fun stuff in that space which is related to different to this both kind of the duality, I think, about

Daniel: a year ago now, at cider that we were we were talking without the existence of this paper quite yet, but the this principle of the you could infer mutual knowledge and basically these types of mechanisms with the same group of folks. So I'm really excited to see what they end up coming up with, because a lot this is very related that you know, seeing the junction of error correcting and inferring knowledge, I've always really appreciated there that the Berkeley cruise perspective on how these are common knowledge problems, there

Pat : are tons of fun. And then my cider paper from January of this year, which was called, you know, scalable OTP in the cloud. What's the big deal? Is all about, how can we reason about a scalable transaction system and detect conflicts on key conflicts and detect when some with an abounding point of time? How can we know recent changes to sets of data? And so all of this is still nascent, and there's tons of tons of future in this it's an exciting time to be a nerd.

Speaker 2 1:03:29 Totally agree, mother, know your name coincides with the mathematician who actually came up with the error encoding in India, like two years back.

Unknown Speaker 1:03:41 Oh, interesting.

Pat : Yeah, that's cool. That's cool. Well, I really want to thank you guys for your attention.

Hesham: I think one of the good things about your presentation is that you use animation.

Unknown Speaker 1:03:58 Oh, that's great to describe

Hesham: what is difficult to stop that you might not be able to get when you read the paper. Like how to understand is a reckless, invertible blue lookup tables, which is the main contribution of his paper, really, you explained it in a way, very good to understand by anybody.

- Pat :** Thank you. I spent many, many, many hours fiddling with PowerPoint, and I just this presentation. I needed two laptops, one for the screenshot of PowerPoint, and so no like but I I believe people's time is worth spend, investing in making their time efficient. And,
Speaker 4 1:04:47 yeah, is this presentation going to be available or
- Pat :** I we have not yet. We're kind of fine tuning it honestly, and learning and adapting it. And I was just right before the hour, talking about this with, okay, Daniel, let's sit down with Dan PowerPoint and put our little faces in the corner of that thing. Literally, when I emptied out, that is empty presentation, I put the box in the corner doing precisely the size of the video of your face. The PowerPoint will record in anticipation of doing that, and then pushing it up on YouTube so it's available. So that's planning it. And because I've been there, done that, got the t shirt. If you want to know the extent of it, go look at my my blog and see that the video of the I'm so glad I uncoordinated from a couple years ago, which had even more sophisticated animations. And so, because I sometimes, when I'm, you know, you fiddle and you're thinking, drawing animations is kind of like fiddling when I'm thinking sometimes, so but it but thank you. I'm glad you enjoyed it. I know I pushed a lot of content in a little window of time,
Speaker 1 1:05:47 yeah, but the animation makes it easier to understand. Thank you, and I agree, yeah, when I looked at the YouTube presentation for the authors they put in the second conference, it wasn't that easy to understand. To be honest with
- Pat :** you, the paper wasn't that easy to understand how I know of really good stuff on Shannon theoretical. And believe me, Daniel and I were arguing about this for months before we knew how to understand it and then explain it. So we had a lot of fun, but it was a lot of arm wrestling between the two of us. Was
Speaker 1 1:06:24 fun, yeah? And supervisor for this experiences, Muhammad Ali, said that he's very, very smart about very
- Pat :** smart guy, and I'm not that ready. Well, great. Actually, trying to come at it from a district systems perspective, a little more than a data communication, but there's a, there's gold in that paper.
Speaker 1 1:06:43 Yeah. And the claim is right, you have the first universal sketch for algorithm. Nobody has done it before, right? That's, that's the claim. And I think you're right.
- Pat :** I want to acknowledge what they've done, but I want a slightly different perspective.
- Daniel:** I think that this is an area where they did an amazing job of connecting the dots, because Ken Berman brought to our attention that there actually might be a few papers that were similar to this, but never achieved the same efficiency that were sort of lost to time. But this is the first, the first time that we've seen the real connection of error correcting code, basically just erasure code research, and what you can do to implement some reconciliation with it and actually achieving those the full target
Speaker 3 1:07:39 efficiency, exactly.
- Pat :** I fully believe that if you have set reconciliation, you can build everything else on top of it. That's just fine.
- Hesham:** I agree. And also they will provide good balance between computation and communication efficiency algorithms. I don't see that. So that is also good contribution,
- Pat :** yep. So I just want to get everybody excited and people do good stuff and let me know if you got some ways to improve on and bounce around, if you want to be a nerd together. I'm a nerd. I want to do nerd shit with people, if that makes any sense. So thank you Amit for including us. We appreciate you.

Ahmad : So next, like next talk will be January 1 will be basically next year. By Paul Borrill, about Ethernet space time. And Paul, do you want to give it a preamble before we close? Or you want to keep everyone surprised?

Paul: Actually, we're announcing Ethernet 2025. We have been working on the Shannon set reconciliation problem for years, and have made some significant progress in the last year.

Really what we're looking at is how the notion of time can be expressed on Ethernet, if you can have essentially different numbers of fixed size objects, the slots that Pat is talking about here basically for just entanglement³. You can do it with four slots. And a two phase commit, for example, you can do it with 16 slots. And to do more advanced multi phase protocols, you can have any arbitrary amount of basically, it's a hard about number of slots.

And so this set reconciliation algorithm fits perfectly with what we've begun to understand from the physics perspective. And just let me give you a little bit of interesting pieces here, one of the things that the quantum guys have actually discovered is this thing called "forgetting", or what we say when we use up the SmartNIC resources "discarding".

Apparently, if you forget things, then you are basically causing the causality to do different strange things to you. And also dropping, if you drop packets, for example, or if you forget things, forgetting and discarding are the two key issues that's being thoroughly investigated right now by both the experimentalists as well as the theoreticians in the physics community.

Pat : What I'm looking for this on New Year's Day, because with being hungover, it should make more sense. But I gotta go right, I'm looking forward to it New Year's Day, let us know.

³Entanglement is Monogamous. It take two to tango, and three to party.